When Locality Does Not Reduce Memory Time: A Trace Study of Graph-Based Nearest-Neighbour Search

Artashes
Computer Science / Computer Architecture (HPC)
Companion paper to the thesis, 9 November 2026

Abstract—Storing frequently accessed data together seems a natural way to speed up graph search. Yet a memory system can benefit both from reusing an open DRAM row and from serving work across different banks. This study examines that trade-off in approximate nearest-neighbour search. It records the vector and graph-list reads of stock HNSW search, changes their vector placement, and measures completed service in a pinned HBM2 simulator. On ten matched SIFT100k query splits, a workload-trained layout changes cycles relative to breadth-first placement by -0.31% at 512 bytes per vector and +0.17% at a hypothetical 128-byte footprint. In the latter case, it achieves more row hits but takes longer. Command counts and component-stream experiments explain how fewer row operations can coexist with more waiting. Repeated controller experiments preserve their conditional signs, while additional graph constructions reverse the 128-byte ranking. The footprint alone does not determine the winner. The contribution is a reproducible characterisation of memory-service behaviour, with practical guidance on metrics and effect size. Computation, caches and an integrated processing-in-memory system are outside the measured model.

Index Terms—approximate nearest neighbours, graph search, memory layout, DRAM simulation, processing-in-memory

I. The engineering question

A vector-search system stores an array of numbers for each item and retrieves items close to a query array under a distance function. Image retrieval and embedding search are examples. An exact scan compares the query with every stored vector. Approximate nearest-neighbour (ANN) methods reduce that work by examining promising candidates, accepting that some exact neighbours may be missed.

Hierarchical Navigable Small World (HNSW) search represents vectors as nodes in a proximity graph [1]. To expand a node, it reads an adjacency list of neighbour IDs, fetches candidate vectors, and evaluates their distances. Those distances guide the next expansion. For example, expanding node 17 might reveal candidates 4 and 29. Their addresses become useful through the earlier graph work; the complete future request sequence is not initially known to a real execution.

A layout can store related nodes close together. This may improve memory service, but "close" is only an address property. The memory controller decides which commands can issue and which requests must wait. We ask three connected questions: how request order affects that service; whether a workload-trained placement improves on simple graph layouts; and whether the answer persists when memory assumptions or graph construction change.

The study measures DRAM stream service: the controller cycles required to complete an explicitly supplied read sequence. This isolates placement from search work. It also means that the measured cycles exclude computation, caches, scratch structures, mapping lookups and the time needed to discover future graph addresses. Processing-in-memory (PIM), which places computation in or near memory, motivates the investigation but is not implemented as an application platform.

II. Why locality can help or hurt

A DRAM bank has a row buffer holding an open row. An activate command (ACT) opens a row, a read command (RD) transfers data from it, and a precharge command (PRE) closes it before another row can open. A request to the already open row is a row hit. Avoiding repeated row changes can save commands and waiting time.

Banks also provide parallel service opportunities. While one bank waits for a timing constraint, another may make progress. Consecutive requests concentrated in one bank can reduce these opportunities. A row hit therefore need not complete immediately: command spacing, shared resources and other requests still matter.

The controller's choices depend on concurrency. With one outstanding read—a request accepted but not completed—there is little opportunity to overlap different reads. With many outstanding reads, bank diversity can be useful. The page policy controls when rows close, the address mapper selects a bank and row from address bits, and the scheduler chooses among queued commands. These are experimentally controlled variables here.

A CPU cache adds another layer: a cache hit can avoid a DRAM request entirely. A DRAM row hit still requires memory service. Keeping that distinction clear is essential when comparing this study with cache-oriented graph reordering.

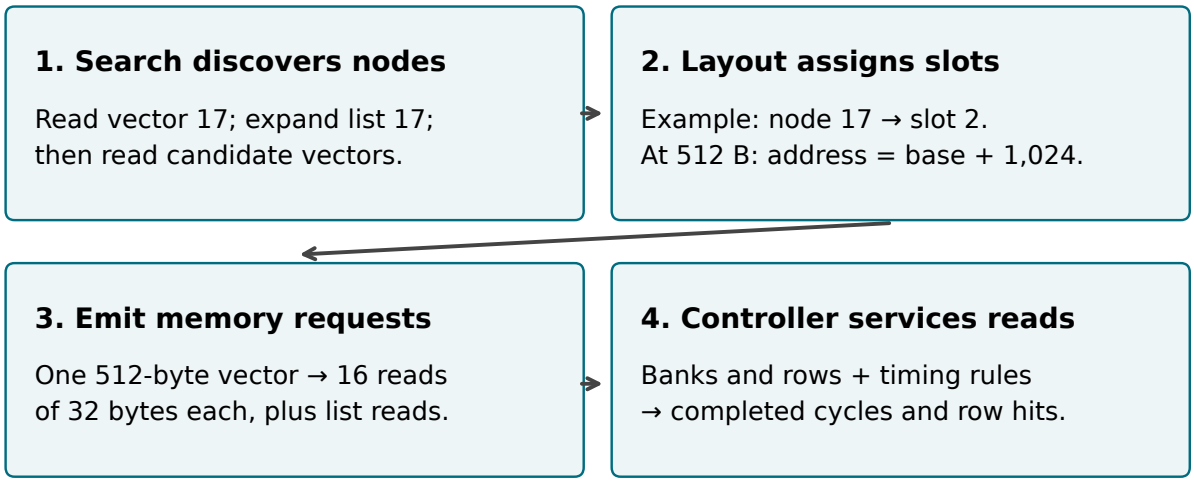


Fig. 1. A logical search determines which data are needed. A layout determines their addresses. The emitter turns payloads into native requests, and the controller determines service time. The node and slot numbers illustrate the transformation.

III. From search to a controlled experiment

A. What stays fixed and what changes

Figure 1 shows the experiment chain. Observer hooks in stock hnswlib record each required vector read and each expanded adjacency list, including upper graph layers and the entry vector. Search decisions and result ordering remain in stock code. The driver verifies every observed result against unobserved search, and tests compare 800 results with independently built indexes, including ties, narrow search settings and shuffled insertion.

A layout assigns every corpus node a unique vector slot. If node v is assigned slot $\pi(v)$, its vector begins at $A_V + \pi(v)b$, where A_V is the array base and b is the stored vector size. All layouts for one graph and split receive the same logical accesses and return the same answers. They change addresses, rather than the number of distance evaluations.

Adjacency lists occupy separate, aligned allocations independent of vector layout. A list containing d neighbour IDs requires $4 + 4d$ bytes: a four-byte header plus four bytes per ID. The emitter covers each payload with the simulator's native 32-byte reads. Thus a 512-byte vector needs sixteen reads and a 128-byte vector needs four. No cache or duplicate-read coalescing is assumed.

B. The five layouts

Identity keeps corpus order; random supplies a shuffled control. Breadth-first search (BFS) placement visits graph neighbours before the next frontier and stores that visitation order. Community placement groups nodes using repeated neighbour-label updates. These two methods use graph structure without training queries.

The query-aware method learns which different vectors are accessed close together in training searches. It counts pairs using a bounded history of 16 distinct nodes within each query. It then greedily places heavily co-visited nodes together, without crossing the current grouping boundary. This can capture shared visits between nodes that have no direct graph edge. It optimises a co-visitation objective, however, rather than simulator cycles.

The verified contiguous same-bank/row span is 1,024 bytes under the evaluated mappings. A fresh aligned block fits two 512-byte vectors or eight 128-byte vectors, so the heuristic is rebuilt for each footprint. The smaller footprint is hypothetical repacking: float32 search decisions, distances and recall remain unchanged. It is not an implementation of quantised HNSW or product quantisation (PQ), which changes representation and distance evaluation [2].

C. Memory model and workload

Ramulator is pinned to revision 7bead6cb7c4b [4], [5]. We configure one HBM2 channel using the HBM2_8Gb and HBM2_2000Mbps presets. The GenericDDR controller selects at most one command per controller cycle. Default settings are an open-page policy, readiness-first FRFCFS scheduling, 64 outstanding reads, a 32-entry read buffer and one issue opportunity per cycle. The bound counts transactions: 64 native reads represent four full 512-byte vectors or sixteen 128-byte vectors before adjacency traffic. Footprint therefore changes both grouping capacity and the amount of logical vector work inside the bound. The full thesis documents address-bit assignments and timing assumptions; integration tests verify the actual mapper and native transaction size.

Each query drains its reads before the next query issues, while open-row state is retained. The frontend waits for every completion. Every replay must reconcile trace reads, submissions, completions, served requests and row classifications. Optional command observers preserve the

baseline timing and record the commands used in the mechanism analysis.

The primary data are the first 100,000 SIFT vectors from the ANN-Benchmarks mirror [7], with 128-dimensional squared Euclidean distance. HNSW uses $M = 16$, construction search width 200, query search width 64 and $k = 10$. These parameters control connectivity, construction effort, search effort and the number of requested answers, respectively. Exact top-ten neighbours are recomputed over this same corpus subset. Returning nine of those ten would give Recall@10 of 90%.

D. Repetition and interpretation of intervals

Each of ten seeded shuffles of the 10,000-query pool supplies 500 training and 200 held-out test queries. They are disjoint within a split. The same test queries evaluate every layout, and graph seed 100 and corpus insertion order are fixed in this primary comparison.

For split s and layout L, the paired cycle difference is

$$d_{s,L} = 100\,(T_{s,L}/T_{s,\mathrm{BFS}} - 1)\,. \tag{1}$$

Negative means fewer cycles than BFS. For example, 99,700 against 100,000 cycles is -0.3%. Pairing removes the confounding effect of testing one layout on more difficult queries. We report the mean difference and a Student 95% interval over the ten split effects, using nine degrees of freedom [14]. It describes uncertainty in this mean under the fixed graph and model, not the spread of individual query latencies or all possible datasets.

Three controller settings are repeated across all ten splits. A separate construction study evaluates graph seeds 101 and 102 in corpus order, and graph seed 100 with insertion-order seed 17, each on splits one through three. External corpus IDs are preserved after shuffling. Means and observed ranges for those selected constructions are reported separately; they are not pooled into a population confidence interval. All comparisons remain exploratory, without a family-wise multiplicity claim.

IV. Results: from the simple case to graph search

A. A controlled reversal and its mechanism

The sequential and permuted microbenchmarks read the same 4,096 vectors, each 128 bytes, for 16,384 transactions. At the default concurrency, sequential input has 96.78% classified row hits but takes 30894 cycles; permuted input has 76.85% hits and takes 28968 cycles. More hits coincide with more time.

Table I separates cycles that issue a command from cycles that do not. Permutation requires more row operations, but at high concurrency it more than compensates through fewer no-command cycles. Its offered stream spans an average of 14.81 distinct bank instances per 64 consecutive requests, compared with 2.97 for sequential order. This is input diversity, not measured queue occupancy. The recorded read-command timeline in the full thesis shows the different bank exposure during service.

TABLE I
Command-level explanation of the controlled ordering experiment. Limit is the maximum number of outstanding reads. ACT opens a row; PRE closes one. "No command" counts controller cycles in which no command issues, including timing waits and final draining. It is not a count of one specific stall cause.

Limit	Order	Cycles	ACT	PRE	No command
64	Sequential	30894	528	38	13930
64	Permuted	28968	3793	3319	5458
1	Sequential	325786	588	166	308482
1	Permuted	398592	4044	1135	376825

With the outstanding limit reduced to one, the result reverses: avoiding row changes becomes more valuable because other reads cannot hide the waiting. These controlled interventions and command records explain a locality–parallelism trade-off without attributing every idle cycle to one timing constraint.

Additional experiments vary disjoint request-sorting windows, contiguous run length, alignment and placement perturbations. Sorting the entire known stream exactly reconstructs sequential input, but dependent graph search may not know those future addresses in advance. The run experiments also distinguish the verified 1-KiB address span from an empirical saturation point, which changes with alignment and the tested grid. Full tables are retained in the thesis and artifact.

B. Placement effects on held-out SIFT queries

Mean Recall@10 is 98.99%, with split-level interval [98.77, 99.22]%. Table II compares layouts at this fixed search operating point. The query-aware layout saves a small fraction of cycles at 512 bytes and costs a small fraction at 128 bytes. The sign is a measured result of each footprint-specific layout and controller interaction.

TABLE II
Mean cycle difference relative to BFS and paired 95% interval over ten query splits. Negative is faster. The 128-byte case changes storage footprint while retaining float32 search.

Layout	512 bytes (%)	128 bytes (%)
Identity	+0.30 [+0.24, +0.36]	+2.68 [+2.60, +2.76]
Random	+0.31 [+0.12, +0.49]	+2.92 [+2.80, +3.04]
BFS	+0.00 [+0.00, +0.00]	+0.00 [+0.00, +0.00]
Community	+0.15 [+0.01, +0.29]	+2.35 [+2.17, +2.53]
Query-aware	-0.31 [-0.37, -0.26]	+0.17 [+0.11, +0.24]

At 128 bytes on split one, query-aware placement improves the row-hit fraction yet takes 2,848 more cycles. Its command record contains 12,516 fewer activations and saves 25,125 command-issue cycles, while adding 27,973 no-command cycles. The reversal remains when adjacency requests are removed. Adjacency-only streams are identical for the two layouts and produce identical timing. These tests locate the effect within vector service

and scheduling interactions; component times cannot be added because mixing traffic changes the schedule.

The distinction between locality within and between vectors also matters. Under an ideal uninterrupted sequence with a first-read miss, the remaining fifteen of sixteen reads of a 512-byte vector could hit the open row: 93.75%. For four reads the analogous fraction is 75%. These are illustrative within-vector opportunities, not predictions of the simulator's hit count. A high aggregate hit rate can therefore coexist with little useful placement of different vectors.

C. How robust is the ranking?

The repeated settings change concurrency, row closure and bank mapping separately. Table III reports the query-aware/BFS differences; all five layouts are retained in the artifact. The query-aware/BFS sign is consistent across all ten splits in each repeated setting: faster at both widths with one outstanding read or the alternate mapper, and slower under cap four. Under alternate mapping, random placement is faster still at 512 bytes. These estimates extend the earlier one-split observations. Because the settings were chosen after inspecting that exploratory matrix and include its first split, this is a robustness extension rather than an independent confirmation.

TABLE III
Selected controller settings repeated across ten paired splits. Other controller and heuristic ablations remain single-split exploratory results.

Setting	Bytes	Δ QA/BFS (%)	95% interval
Outstanding: 1	512	-0.13	[-0.14, -0.13]
Outstanding: 1	128	-1.26	[-1.30, -1.21]
ClosedCAP, cap 4	512	+0.89	[+0.79, +0.99]
ClosedCAP, cap 4	128	+5.27	[+5.15, +5.39]
Alternate mapper	512	-4.02	[-4.28, -3.75]
Alternate mapper	128	-4.12	[-4.26, -3.99]

The additional graph constructions are compared on matching query splits in Figure 2. At 512 bytes, query-aware is faster than BFS in every tested construction/split pair. At 128 bytes, the original graph has a three-split mean of +0.119%, compared with -0.191% and -0.705% for graph seeds 101 and 102. Shuffled insertion gives splits on both sides of zero. The original 128-byte loss therefore does not generalise even across these graph constructions. Changing the graph can also change the search work and recall, so the cycle ratios always compare layouts within that graph. Recall across the evaluated graph/split conditions ranges from 98.55% to 99.35%.

Training-size, history-length and grouping-size ablations expose the heuristic's dependence on its training objective. They are not an exhaustive tuning search. Default training covers 97.7% of test vector accesses by node occurrence. High node coverage alone does not show that useful pair relationships or controller opportunities were learned.

V. What an engineer should take from the results

The default float32 advantage over BFS is approximately 97.3 simulated controller cycles per query. This provides a scale for judging proposed implementation costs. Under an illustrative additive model, any extra per-query overhead of that size would erase the advantage. CPU preprocessing seconds cannot be divided by simulated controller cycles to infer a real deployment break-even point.

The two stored 64-bit mapping directions require 1.53 MiB for 100,000 nodes, excluding container overhead. A deployed implementation may use a different representation, but its translation, storage movement and rebuilding costs need measurement. Shared-host preprocessing timings are retained for accounting; they are not isolated platform benchmarks.

The practical evaluation sequence is: hold answer quality and logical work fixed; measure completed time as well as locality; check the mapper and realistic concurrency; and compare the gain with the costs of integrating the layout. Simple graph placement is a useful baseline. The best measured layout under one controller is a candidate for further evaluation, not a universal recommendation.

VI. Position relative to prior work and limits

Coleman et al. study HNSW graph reordering, cache misses and query time, reporting improvements up to 40% in their evaluation [8]. The present study examines uncached DRAM demands and controller interactions, so its smaller effects address a different part of the execution path. It neither reproduces that study nor establishes a better graph-reordering algorithm. DiskANN further illustrates that search performance depends on a complete storage-system design [9].

PIM research combines placement with computation and communication [3]. DRIM-ANN includes arithmetic lookup tables, load balancing and I/O optimisation; its headline speedups concern computationally enhanced PIM [10]. UpANNS evaluates commercial-PIM resource management and ANN co-design [11]; PIMANN studies persistent kernels and fine-grained scheduling [12]. The 2026 PIMCQG preprint directly addresses graph ANN on PIM with a compact index and asynchronous execution [13]. This thesis contributes a controlled memory-service characterisation within that broader systems problem.

The evidence covers one real corpus prefix, selected graph constructions, one HBM2 model and fixed ANN parameters. The smaller footprint does not implement quantisation. Flat replay knows future accesses earlier than a dependent execution may discover them. Caches, computation, scratch state, host/PIM transfers, energy, workload drift and actual mapping costs remain outside the evaluation. The simulator preset itself is not validated against a physical HBM2 device here.

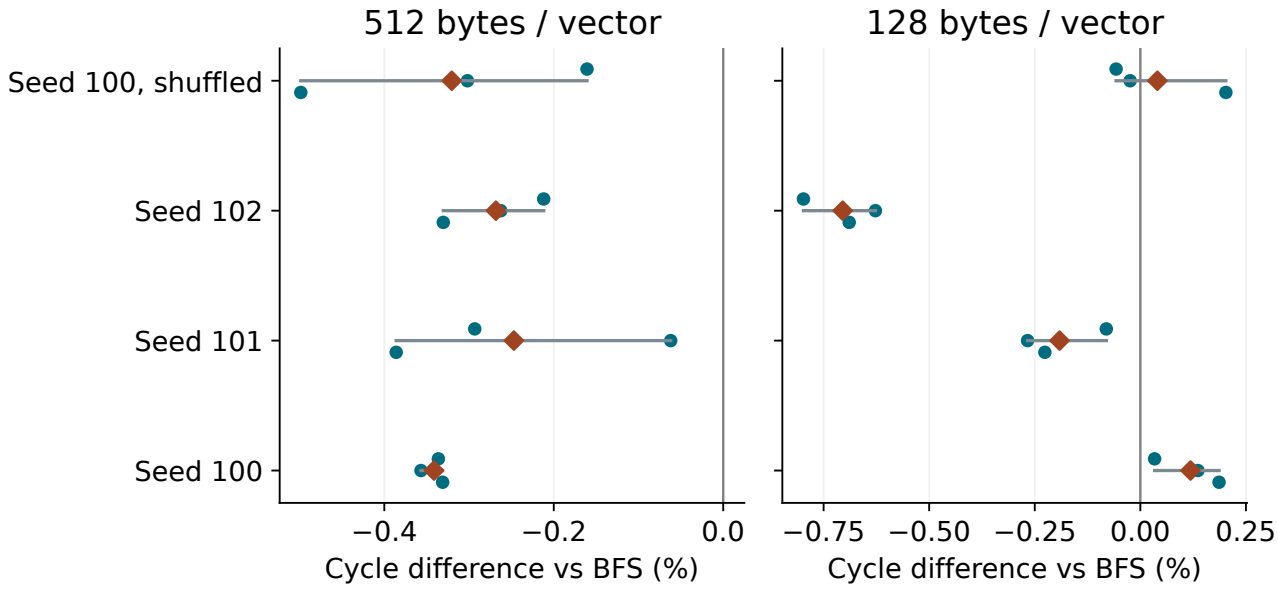


Fig. 2. Query-aware/BFS differences for three matching query splits per construction. Teal points are split effects, the orange diamond is their mean, and the grey line spans the observed range. These are descriptive ranges, not confidence intervals over graphs.

VII. Conclusion

Vector placement affects memory service through both row reuse and the work exposed to the controller. The measured ranking can reverse even while the row-hit metric improves. Changing graph construction also reverses the 128-byte ranking under the same default controller. Paired workload comparisons quantify the small default effect; repeated controller settings, construction variation and command-level diagnostics show how to interpret it. The result is useful as an evaluation method and a bounded empirical characterisation. An application-speedup claim requires the next layer of evidence: a complete execution path with its computation, dependencies and integration costs.

References

[1] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824–836, 2020. https://doi.org/10.1109/TPAMI.2018.2889473.

[2] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 33, no. 1, pp. 117–128, 2011. https://doi.org/10.1109/TPAMI.2010.57.

[3] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungnirun, "A modern primer on processing in memory," in Emerging Computing: From Devices to Systems, Springer, 2022. https://arxiv.org/abs/2012.03112.

[4] H. Luo et al., "Ramulator 2.0: A modern, modular, and extensible DRAM simulator," IEEE Computer Architecture Letters, 2023. https://arxiv.org/abs/2308.11030.

[5] CMU-SAFARI, Ramulator2 source, revision 7bead6cb7c4b (full revision in link), inspected 9 November 2026. https://github.com/CMU-SAFARI/ramulator2/tree/7bead6cb7c4b03e095c6d4769ae3181a8e0bf6da.

[6] nmslib contributors, hnswlib: Header-only C++/Python library for fast approximate nearest neighbors. Vendored source and observer patch identified in the accompanying artifact. https://github.com/nmslib/hnswlib.

[7] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," Information Systems, vol. 87, 2020. Dataset mirror: https://ann-benchmarks.com/sift-128-euclidean.hdf5.

[8] B. Coleman, S. Segarra, A. Shrivastava, and A. Smola, "Graph reordering for cache-efficient near neighbor search," arXiv:2104.03221, 2021. https://arxiv.org/abs/2104.03221.

[9] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnaswamy, and R. Kadekodi, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," Advances in Neural Information Processing Systems, 2019. https://github.com/microsoft/DiskANN.

[10] M. Chen et al., "DRIM-ANN: An approximate nearest neighbor search engine based on commercial DRAM-PIMs," SC, 2025. https://doi.org/10.1145/3712285.3759801; preprint https://arxiv.org/abs/2410.15621.

[11] S. Chen, A. C. Zhou, Y. Shi, Y. Li, and X. Yao, "UpANNS: Enhancing billion-scale ANNS efficiency with real-world PIM architecture," SC, 2025. https://arxiv.org/abs/2410.23805.

[12] P. Wu, M. Xie, E. Zhao, D. Zhang, J. Wang, X. Liang, K. Ren, and Y. Chai, "Turbocharge ANNS on real processing-in-memory by enabling fine-grained per-PIM-core scheduling," USENIX Annual Technical Conference, 2025. https://www.usenix.org/conference/atc25/presentation/wu-puqing.

[13] S. Chen, Y. Li, Y. Chen, M. Deng, J. Meng, and A. C. Zhou, "PIMCQG: Co-designing billion-scale graph-based approximate nearest neighbor search for processing-in-memory," arXiv:2605.25522, first posted 25 May 2026, version updated 3 November 2026, accessed 9 November 2026. https://arxiv.org/abs/2605.25522.

[14] Student, "The probable error of a mean," Biometrika, vol. 6, no. 1, pp. 1–25, 1908. https://doi.org/10.1093/biomet/6.1.1.